Guide on How to Submit the Assignment

Overview

This guide provides the steps to submit your smart contract assignments using GitHub. Follow these instructions to ensure your submission meets all requirements.

Submission Steps

1. Complete All Assignment Tasks
    - Ensure you have written and tested all required smart contracts in Solidity. Save each smart contract file using the format `q1.sol`, `q2.sol`, etc.
2. Save Your Work
    - Save each smart contract in separate files, e.g., `q1.sol`, `q2.sol`.
    - Ensure each file is properly formatted and free of syntax errors.
3. Add Comments
    - Add comments in each Solidity file to explain your logic and steps. This will help reviewers understand your thought process.
    - Ensure each function and significant section has a brief description at the top.
4. Set Up Your GitHub Repository
    - Create a GitHub Repository:
        1. Go to GitHub and log in to your account.
        2. Click on the "New repository" button.
        3. Name your repository using the format: `YourGitHubUsername_AssignmentRepo`.
        4. Initialize the repository with a README file.
    - Upload Your Files:
        1. Clone your repository to your local machine.
        2. Create a new folder for each assignment inside your cloned repository. For example, create folders like `assignment1`, `assignment2`.
        3. Save your Solidity files (`q1.sol`, `q2.sol`, etc.) and `README.md` into the corresponding assignment folder.
        4. Commit your changes and push them to the repository.
5. Submit Your Assignment
    - In the Classroom:
        1. Log in to the classroom platform.
        2. Navigate to the assignment submission section.
        3. Submit the GitHub repository link where your code and documentation are hosted.

6. Verify Submission
    - Ensure your GitHub repository is publicly accessible.
    - Double-check that all required files are included in the repository and properly organized.
    - Confirm that you have submitted the correct repository link in the classroom platform.

Important Notes

- Deadlines: Ensure you submit before the deadline to avoid penalties.
- File Naming: Adhere to naming conventions for easy identification.
- Code Quality: Write clean, readable, and efficient code.

1. **Hello World Contract:**
    a. **Question:** How can a smart contract return a simple message, like "Hello, World!"?
2. **Simple Storage:**
    a. **Question:** How can a smart contract store and retrieve a single integer value?
3. **Greeting Contract:**
    a. **Question:** How can a smart contract allow a user to set and get a personalized greeting message?
4. **Simple Counter:**
    a. **Question:** How can a smart contract keep track of a count and allow it to be incremented?
5. **Name Storage:**
    a. **Question:** How can a smart contract store and retrieve a user's name?
6. **Basic Voting:**
    a. **Question:** How can a smart contract allow users to vote for a candidate and keep track of votes?
7. **Owner Access:**
    a. **Question:** How can a smart contract restrict certain functions to only the contract owner?
8. **Event Logging:**
    a. **Question:** How can a smart contract log events when certain actions are performed?
9. **Simple Ledger:**
    a. **Question:** How can a smart contract maintain a ledger of transactions with basic entries?
10. **Message Storage:**
    a. **Question:** How can a smart contract allow a user to store and retrieve a message string?